

SPATIAL VERSUS TEMPORAL EDGE DETECTION: 2D CONVOLUTION AND EVENT-BASED SIMULATION

Wenyin (Knut) Gu

1. Background and Motivation

In EECS 150 class, we learned how filtering allows us to extract specific frequency components from a signal. One idea that particularly struck with me is “edges” in images are fundamentally high-frequency components. Traditional image processing methods make this insight concrete by applying spatial convolution kernels (such as Sobel operators) to detect edges in static images.

However, recent research in Event-based Vision [Source: 2, 3, 4] shows a limitation of this approach: it processes the entire image frame, including redundant static backgrounds. Bio-inspired event cameras solve this by responding only to *changes over time*, thereby effectively detecting temporal edges and ignoring redundant background information.

What motivates this project is my curiosity about whether these two seemingly different approaches—spatial edge detection and temporal event-based detection—can be understood through the same signals-and-systems framework that we are learning in class. The aim of the project, therefore, is to demonstrate that both methods are essentially high-pass filters, but they operate in different domains: space versus time. This project is worth pursuing because it allows me not only to deepen my understanding of core EECS 150 concepts but also to see how the classical signal processing ideas can be extended to contemporary computer vision problems.

2. Problem Definition

This project is framed as a comparative study of two Linear Time-Invariant (LTI) systems that perform edge detection in fundamentally different domains.

System A: Spatial Filtering

- ▶ Input: A single static grayscale image $x[n_1, n_2]$, where n_1 and n_2 denote spatial coordinates.
- ▶ System: A 2D Convolution system ($y = x * h$), where the impulse response h is a standard edge-detection kernel (e.g., Sobel).
- ▶ Goal: To extract structural outlines of all objects in the scene.

System B: Temporal Filtering

- ▶ Input: A sequence of two consecutive image frames, $x[t]$ and $x[t - 1]$.
- ▶ System: A discrete-time difference equation defined by $y[t] = |x[t] - x[t - 1]|$.
- ▶ Goal: To simulate the behavior of an Event Camera (DVS). This acts as a temporal high-pass filter that suppresses static backgrounds and only extracts “active” edges (i.e., moving objects).

By placing these two systems side by side, the project asks the following guiding question: how do spatial and temporal high-pass filtering differ in what they extract (or what they ignore) when applied to visual data?

3. Methods

Programming Language:

The project is implemented in Python, using NumPy (for matrix operations) and Matplotlib (for visualization).

System A: Spatial Filtering

I implement 2D convolution using `scipy.signal.convolve2d` and apply Sobel-type kernels to static images.

To detect vertical edges, I apply the horizontal Sobel kernel h_x using 2D convolution. Consider a 3x3 pixel region x where a brightness jump occurs:

$$x = \begin{bmatrix} 10 & 10 & 200 \\ 10 & 10 & 200 \\ 10 & 10 & 200 \end{bmatrix}, h_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

The output g_x is calculated as an element-wise product and sum:

$$g_x = x * h_x = (10 * -1) + (10 * 0) + (200 * 1) + (10 * -2) + (10 * 0) + (200 * 2) + (10 * -1) + (10 * 0) + (200 * 1) = 760$$

This high value represents a detected edge (i.e., white pixel).

While g_x detects vertical edges by measuring horizontal changes, the system also calculates the vertical gradient g_y using the kernel h_y :

$$h_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

By applying h_y to the same pixel region, the system identifies horizontal edges. The final edge intensity for each pixel is then determined by combining both gradients, typically using the magnitude formula $G = \sqrt{g_x^2 + g_y^2}$. This ensures that edges at any orientation are captured in the final spatial output.

System B: Temporal Filtering

I implement the pixel-wise difference equation. To make the comparison meaningful, I design test cases such as a moving object against a textured but static background to visually demonstrate the advantage of temporal high-pass filtering in background suppression.

This system implements the equation $y[t] = |x[t] - x[t - 1]|$. To simulate motion, I compare a pixel at $t - 1$ (background) to t (object moved in):

$$x[t - 1] = [50], x[t] = [200] \Rightarrow y[t] = |200 - 50| = 150$$

If the area is static background ($50 \rightarrow 50$), the result is 0, demonstrating perfect background suppression.

To properly evaluate the temporal difference equation, I designed a local motion simulation, as illustrated in Fig. 1. Instead of shifting the entire image, I defined a specific Region of Interest (ROI) containing the bottle. Between frame $t - 1$ and frame t , only the pixels within this ROI were shifted horizontally, while the rest of the image, representing the table and the background, remained static. This experimental setup isolates the motion of the target object, allowing the system to test whether the temporal filter can successfully suppress redundant background data.

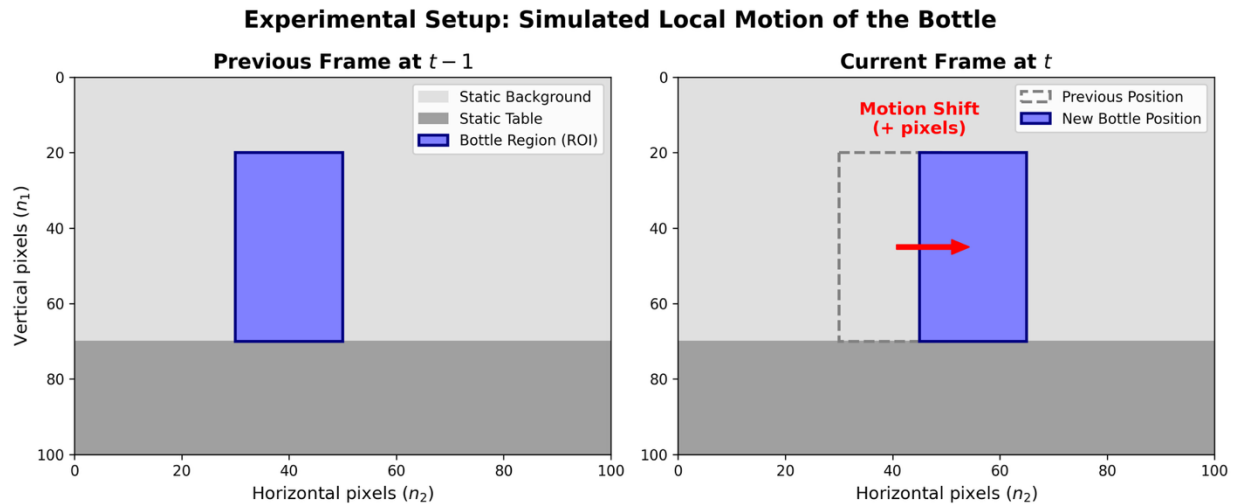


Fig. 1: Local motion simulation.

Only the target object (bottle) is shifted horizontally, while the background remains static.

4. Results and Analysis

The experimental results demonstrate a clear distinction between the two LTI systems:

System A: Spatial Filtering

The Sobel operator successfully captured all high-frequency spatial components in the image. As shown in the result, the system produced sharp outlines for the water bottle. However, because this is a spatial-only filter, it also extracted the complex grain of the wooden table and the grainy texture of the ground. While technically accurate for edge detection, this output contains a high amount of “spatial noise” or redundant data if the goal is only to track the object’s movement.

System B: Temporal Filtering

By implementing the discrete-time difference equation and simulating a local motion model, System B achieved a much cleaner and more focused result. The static background, including the table and floor, was effectively suppressed to a value of zero, appearing perfectly black in the output. To further refine this process, a threshold of 30 was applied to filter out minor variations and simulation noise, ensuring that only significant brightness changes caused by the movement of the bottle were recorded. This combined approach makes the final image much clearer, leaving only the sharp “event-driven” edges of the moving bottle while completely removing unwanted background textures.

Signal Sparsity:

As we can see from the experimental demonstration, System B resulted in a much sparser signal than System A. While System A processed *every* pixel's spatial gradient, System B only produced non-zero output where the “event”, i.e., the bottle's motion, occurred.

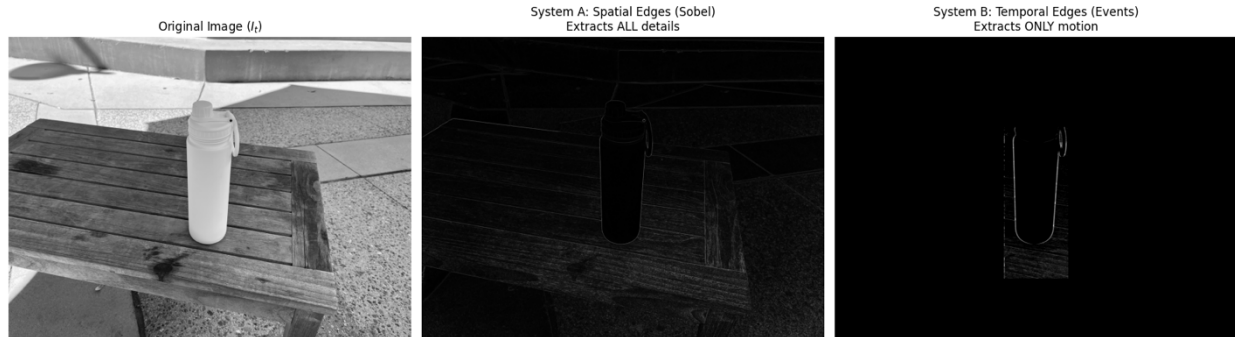


Fig. 2: Comparison of Spatial vs. Temporal Edge Detection.

Left: Original grayscale input; Center: System A (Sobel) extracting all spatial textures; Right: System B (Temporal) isolating only the moving object through simulated events.

5. Discussions

This comparison bridges abstract LTI theory with modern vision technology.

Simulation Artifacts

It is important to note that some residual textures from the table are still visible within the motion region of System B. This occurs because the current experiment uses a simulated local motion model. In this simulation, I defined a specific rectangular Region of Interest (ROI) and shifted all pixels within that box to represent movement. Consequently, any static background textures caught inside that moving box are also shifted, causing the system to detect them as “changes”. In a real-world scenario with a physical moving object and a stationary camera, the background would remain truly static while only the object's pixels change position. Therefore, these residuals are a side effect of the simulation method rather than a flaw in the temporal difference theory itself.

Efficiency and Data Sparsity

The results prove that temporal edge detection is significantly more efficient for motion-tracking tasks.

- ▶ **Redundancy in System A:** In System A, the 3x3 convolution must be performed on every single pixel in every frame. As shown in my results, it processes and generates signals for every static wood grain on the table, creating a massive amount of redundant data that a tracking algorithm would then have to filter out.
- ▶ **Sparsity in System B:** In contrast, System B ignores these static, high-frequency spatial “noise” components because their temporal derivative is zero. By only triggering an output when a pixel's intensity changes, the system drastically reduces the number of active pixels. In a real-world application, this means the processor only handles “events”, i.e., motion, which saves power and reduces latency.

Overall, this project connects abstract Signals and Systems theory to modern computer vision by implementing and comparing two LTI systems: spatial convolution and temporal differentiation. By treating the “event-based” approach as a system with a zero at the origin, I can explain why it functions as a temporal high-pass filter that efficiently suppresses static redundancy. This comparison makes it easy to see how ideas like frequency response translate directly into “edges” and “motion” in the real world. More broadly, this project shows that the convolution and transforms we learned in class are not just theoretical tools, but also the building blocks for understanding—or even improving—emerging approaches to event-based vision.

References

- [1] Phillips, Charles L, et al. *Signals, Systems, and Transforms*. Boston, Pearson, 2014.
- [2] G. Gallego *et al.*, "Event-based vision: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, Jan. 2022.
- [3] D. Tedaldi, G. Gallego, E. Mueggler, and D. Scaramuzza, "Feature detection and tracking with the dynamic and active-pixel vision sensor (DAVIS)," in *Proceedings of the IEEE International Conference on Event-Based Control, Communication, and Signal Processing (EBCCSP)*, Krakow, Poland, 2016, pp. 1–7.
- [4] Y. Zhou, G. Gallego, and S. Shen, "Event-based stereo visual odometry," *IEEE Transactions on Robotics*, vol. 37, no. 5, pp. 1433–1450, Oct. 2021.
- [5] "Sobel operator," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Sobel_operator. [Accessed: Feb. 07, 2026].
- [6] Google, "Gemini 3.1 Pro," Large Language Model, 2026. [Online]. Available: <https://gemini.google.com>